

LAB 4 – VIBECODING

AI-Assisted Software Development Conversation Record

Student: Bhuvu Sudheendra Katti

Course: Software Engineering / VibeCoding Lab

Project: 55 Card Predictor

GitHub Repository: https://github.com/stacktrace-bhuvi/55_card_predictor

Note: This document reconstructs the relevant development workflow from the available project records, task summaries, commits, and test results. It is not a verbatim export of the original AI conversation.

1. Task 1 – Fixing Card Rank Comparison

Student Prompt

The game has a problem with comparing card ranks. Card ranks should be compared according to their actual playing-card values rather than alphabetically as strings.

Inspect the existing implementation and fix the comparison logic. Keep the change focused on the bug and avoid unnecessarily modifying unrelated parts of the project.

AI Response

The issue is caused by comparing card ranks as strings instead of comparing their numerical values.

String comparison does not represent the correct order of playing cards. For example, comparing values alphabetically can produce incorrect results when ranks contain numbers or face-card labels.

The comparison logic should convert the card ranks into their appropriate numerical values:

- Number cards retain their corresponding values.
- Jack is represented as 11.
- Queen is represented as 12.
- King is represented as 13.
- Ace is represented as 14.

The game should then compare the numerical values to determine which card has the higher rank.

The change should be implemented in the existing game logic while preserving the rest of the application's behaviour.

Student Follow-up

After implementing the change, verify that the comparison logic works correctly and commit the fix separately.

AI Response

The card rank comparison bug was fixed by using numerical rank values.

Git commit: 6d6d484

Commit message: Fix card rank comparison bug

2. Task 2 – Implementing Consecutive Win Streak Multipliers

Student Prompt

Add a consecutive-win streak feature to the card prediction game.

When the player makes a correct guess, their streak should increase and the points awarded should depend on the streak. A wrong guess should reset the streak.

Implement this feature without changing unrelated game functionality. Test the logic and keep this task in a separate commit.

AI Response

The game should maintain a streak counter that tracks consecutive correct guesses.

The intended behaviour is:

1. A correct guess increases the streak.
2. The points awarded increase with the streak multiplier.
3. The first consecutive correct guess awards one point.
4. The second consecutive correct guess awards two points.
5. The third consecutive correct guess awards three points.
6. A wrong guess resets the streak.

This makes consecutive correct predictions more rewarding while maintaining a simple scoring system.

The streak and score updates should be integrated with the existing game engine. The implementation should avoid unnecessary changes to the user interface or unrelated game logic.

Student Follow-up

Verify the scoring behaviour for consecutive correct guesses and confirm that a wrong guess resets the streak. Commit this feature independently.

AI Response

The consecutive-win streak multiplier feature was implemented.

Git commit: df0191e

Commit message: Add consecutive win streak multipliers

3. Task 3 – Handling Tied Card Ranks

Student Prompt

Handle the situation where the previous card and the newly drawn card have the same rank.

A tie should be treated as a push rather than as an ordinary win or loss. The game should display an appropriate message and preserve the player's score and streak.

Implement and test this behaviour without changing unrelated functionality.

AI Response

The game should explicitly check whether the two card ranks are equal.

The result should have three possible outcomes:

- **Win:** The prediction is correct according to the game rules.
- **Loss:** The prediction is incorrect according to the game rules.
- **Push:** The two cards have the same rank.

When a tie occurs, the game should display a push message. The player's score and current win streak should remain unchanged.

Handling ties separately prevents them from being incorrectly treated as ordinary wins or losses.

Student Follow-up

Verify that equal card ranks produce the push result and that the score and streak remain unchanged.

AI Response

Tie handling was implemented so that equal card ranks result in a push.

Git commit: 705c638

Commit message: Handle tied card ranks as push

4. Task 4 – Side-by-Side Card Reveal Animation

Student Prompt

Improve the card reveal experience by displaying the previous card and the newly drawn card side by side briefly after a guess.

The cards should remain visible together for approximately 1.2 seconds before the game returns to its normal layout.

Implement this using a non-blocking approach so that the game remains responsive. Prevent additional clicks while the reveal animation is active. Avoid changing unrelated features.

AI Response

The card reveal can be implemented using a timed display state.

After a guess, the game should temporarily display the previous and newly drawn cards side by side. After approximately 1.2 seconds, the display should return to the normal single-card layout.

A non-blocking timing mechanism, such as `pygame.time.get_ticks()`, can be used instead of pausing the program with a blocking delay.

The game should also suppress additional player clicks while the reveal is active. This prevents repeated input from interfering with the transition.

The implementation should preserve the existing game loop, scoring system, and prediction logic.

Student Follow-up

Test the side-by-side reveal, verify that it returns to the normal layout, and confirm that additional clicks are suppressed during the reveal. Commit this change separately.

AI Response

The side-by-side card reveal animation was implemented using a non-blocking timer.

Git commit: 55a5d49

Commit message: Add side-by-side card reveal animation

6. Testing and Verification

Student Prompt

After implementing all the required changes, run the available tests and verify that the game still starts correctly. Check that the changes work together and that no unnecessary modifications have been introduced.

AI Response

The available unit tests were run using:

```
python -m unittest discover -s tests
```

The test run completed successfully, with **6 tests passing**.

The application was also started using:

```
python main.py
```

The recorded development results indicate that the game started without exceptions and that the updated behaviours were checked.

The verification covered the numerical card rank comparison, consecutive-win scoring, tie handling, and side-by-side card reveal.

7. Final Outcome

The project was updated with four main changes:

1. Numerical card rank comparison.
2. Consecutive-win streak multipliers.
3. Push handling for tied card ranks.
4. A timed side-by-side card reveal animation.

The available unit tests passed, the game was reported to start without exceptions, and the changes were organised into separate Git commits and pushed to the personal GitHub repository.

The remaining submission task is to organise the required videos, updated code, and this reconstructed conversation record in the personal Lab repository, then verify the final submission requirements.
